

Assignment-13.4

Name:P.Chithra

Hall no:2303A52137

Question:

Task 1: Refactoring Data Transformation

Logic

Scenario

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

Task Description

- Review the legacy code that computes transformed values using an explicit loop.
- Use an AI tool to suggest a more Pythonic refactoring approach.
- Refactor the code using list comprehensions or helper functions while preserving the output.

Legacy Code

```
values = [2, 4, 6, 8, 10]
```

```
doubled = []
```

```
for v in values:
```

```
    doubled.append(v * 2)
```

```
print(doubled)
```

Expected Output

```
[4, 8, 12, 16, 20]
```

Code:

```
... .py C:\...\python codes from 51 video code.py #generate a python code for student rec.p
D: > Downloads > full stack > Refactor.py > ...
1 values = [2, 4, 6, 8, 10]
2 doubled = [v * 2 for v in values]
3 print(doubled)
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER
Active code page: 65001
D:\Downloads\Python_FullCourse>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d
:/Downloads/full_stack/Refactor.py"
[4, 8, 12, 16, 20]
D:\Downloads\Python_FullCourse>
```

Explanation:

- ❑ The refactored code uses **list comprehension**, which is a more Pythonic way of writing loops.
- ❑ It improves **readability** because the logic is clear and written in a single line.
- ❑ It reduces the number of lines compared to the traditional loop.
- ❑ The code becomes **concise and clean**, making it easier to understand.
- ❑ It avoids creating an empty list and using the append method.
- ❑ List comprehensions are **faster** because Python optimizes them internally.
- ❑ It follows Python's best practices for simple data transformations.
- ❑ The approach makes the program **more efficient and structured**.
- ❑ It is easier to maintain and debug in the future.

🔗 The refactored version produces the **same output** while improving overall code quality.

Task 2:

Improving Text Processing Code

Readability

Scenario

You are working on a log message generator that builds messages word by word.

Task Description

- Analyze the legacy code that constructs a sentence using repeated string concatenation.
- Ask AI to suggest a more efficient and readable approach.
- Refactor the code accordingly while keeping the final output unchanged.

Legacy Code

```
words = ["Refactoring", "with", "AI", "improves",  
"quality"]  
message = ""  
for w in words:  
    message += w + " "  
print(message.strip())
```

Expected Output

Refactoring with AI improves quality

Code:

```
D: > Downloads > full stack > Refactor.py > ...
1 words = ["Refactoring", "with", "AI", "improves", "quality"]
2 message = " ".join(words)
3 print(message)
```

Output:

```
D:\Downloads\Python_FullCourse>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe
:/Downloads/full_stack/Refactor.py
Refactoring with AI improves quality
```

Explanation:

- ◆ First, we create a **list called words**.

This list contains five separate words:

Refactoring, with, AI, improves, quality.

- ◆ Next, we use the **join() function**.

It joins all the words in the list into one sentence.

The " " (space) inside join means a space will be added between each word.

- ◆ The result is stored in a variable called **message**.
- ◆ Finally, the **print() function** displays the complete sentence on the screen.

Task 3:

Safer Access to Configuration Data

Scenario

You are maintaining a configuration loader where dictionary keys may or may not exist depending on deployment settings.

Task Description

- Review the legacy code that manually checks for dictionary keys.
- Use AI suggestions to refactor the code using safer dictionary access methods.

- Ensure the behavior remains the same for missing keys.

Legacy Code

```
config = {"host": "localhost", "port": 8080}
```

```
if "timeout" in config:
```

```
    print(config["timeout"])
```

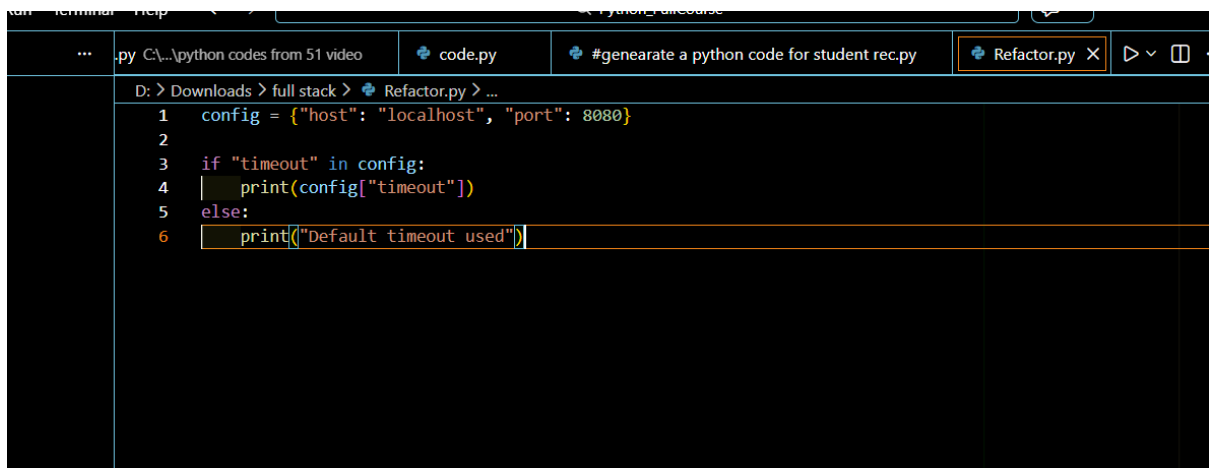
```
else:
```

```
    print("Default timeout used")
```

Expected Output

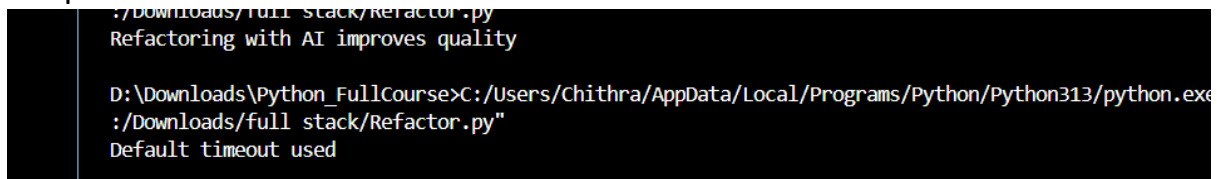
Default timeout used

Code:

A screenshot of a code editor window titled 'Refactor.py'. The editor shows the following Python code:

```
1 config = {"host": "localhost", "port": 8080}
2
3 if "timeout" in config:
4     print(config["timeout"])
5 else:
6     print("Default timeout used")
```

Output:

A screenshot of a terminal window showing the execution of the Python script. The output is:

```
D:\Downloads\Python_FullCourse>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe
:/Downloads/full_stack/Refactor.py"
Default timeout used
```

Explanation:

First, a **dictionary named config** is created.

It stores some settings like **host** and **port**.

◆ Then the program checks whether the key **"timeout"** is present in the dictionary.

◆ The condition

if "timeout" in config

means:

👉 "Check if the timeout setting exists."

- ◆ In this case, the dictionary does **not** contain the key "timeout".
- ◆ So the else block runs and prints:

👉 **"Default timeout used"**

◆ This means the program will use a default value because no timeout is given.

Task 4:

Refactoring Conditional Logic for

Scalability

Scenario

You are enhancing a utility module that performs different operations based on user input.

Task Description

- Examine the multiple if–elif conditions used to determine operations.
- Ask AI to suggest a cleaner, scalable alternative.
- Refactor the logic using mapping techniques while preserving functionality.

Legacy Code

```
action = "divide"
```

```
x, y = 10, 2
```

```
if action == "add":
```

```
    result = x + y
```

```
elif action == "subtract":
```

```
    result = x - y
```

```
elif action == "multiply":
```

```
result = x * y
```

```
elif action == "divide":
```

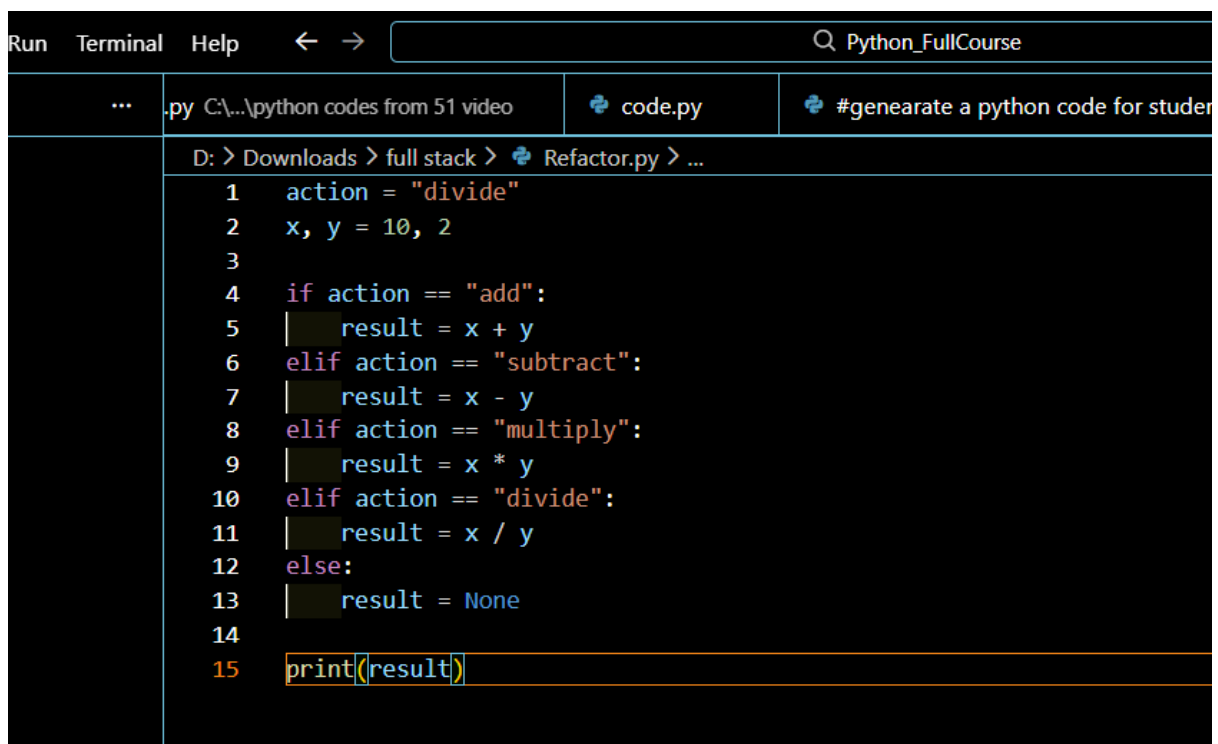
```
result = x / y
```

```
else:
```

```
result = None
```

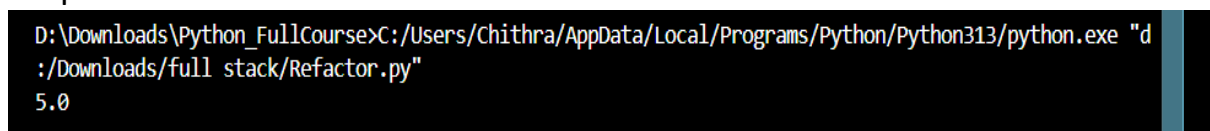
```
print(result)
```

code:

A screenshot of a code editor window. The title bar shows 'Run', 'Terminal', 'Help', and a search bar with 'Python_FullCourse'. Below the title bar, there are tabs for '.py C:\...\python codes from 51 video', 'code.py', and '#generate a python code for studer'. The main editor area shows a Python script with the following code:

```
1 action = "divide"
2 x, y = 10, 2
3
4 if action == "add":
5     result = x + y
6 elif action == "subtract":
7     result = x - y
8 elif action == "multiply":
9     result = x * y
10 elif action == "divide":
11     result = x / y
12 else:
13     result = None
14
15 print(result)
```

output:

A screenshot of a terminal window. The command prompt shows the path 'D:\Downloads\Python_FullCourse' followed by the command 'C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/Downloads/full_stack/Refactor.py"'. The output of the command is '5.0'.

Explanation:

- ◆ First, the variable **action** is set to "divide".

This tells the program which operation to perform.

- ◆ The values **x = 10** and **y = 2** are given.

- ◆ The program checks the value of action using **if–elif conditions**.

It compares the action with different options like add, subtract, multiply, and divide.

- ◆ Since the action is "divide", the program performs **10 ÷ 2**.
- ◆ The result is stored in the variable **result**.
- ◆ Finally, `print(result)` displays the output.

Task 5:

Simplifying Search Logic in

Collections

Scenario

You are reviewing legacy inventory-check code that uses manual loops to find elements.

Task Description

- Identify the explicit loop used for searching an item.
- Use AI assistance to refactor the logic into a more concise and readable form.
- Maintain the same output behavior.

Legacy Code

```
inventory = ["pen", "notebook", "eraser", "marker"]
```

```
found = False
```

```
for item in inventory:
```

```
    if item == "eraser":
```

```
        found = True
```

```
    break
```

```
print("Item Available" if found else "Item Not Available")
```

Expected Output

Item Available

-

Code:

```
D: > Downloads > full stack > Refactor.py > ...
1  inventory = ["pen", "notebook", "eraser", "marker"]
2
3  found = False
4
5  for item in inventory:
6      if item == "eraser":
7          found = True
8          break
9
10 print("Item Available." if found else "Item Not Available.")
11
```

Output:

```
D:\Downloads\Python_FullCourse>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe
:/Downloads/full_stack/Refactor.py"
Item Available.
```

Explanation:

First, a list called **inventory** is created.

It contains items: pen, notebook, eraser, and marker.

- ◆ A variable **found** is set to **False**.

This means the item is not found initially.

- ◆ The **for loop** checks each item in the list one by one.

- ◆ When the program finds "eraser", it changes found to **True**.

Then it stops the loop using break.

- ◆ Finally, the print statement checks the value of found.

If it is True, it prints "**Item Available**", otherwise it prints "**Item Not Available**".

